

Disk Scheduling

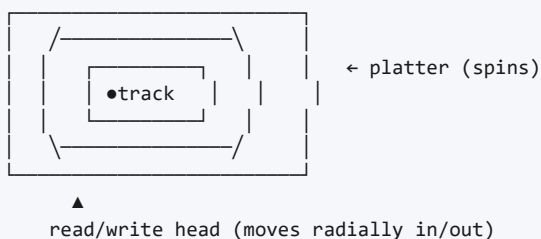
Note: Like File Systems and I/O Management, this topic isn't in your source notes — drafted fresh from general SDE-interview OS knowledge, in the same style as the other topics.

1. Why Disk Scheduling Exists

On a mechanical (spinning) hard disk, the dominant cost of any I/O request isn't transferring the data itself — it's **physically moving the read/write head** to the right location first. Multiple processes issue disk requests concurrently, and the OS gets to choose the **order** in which to service them. Choosing well can save enormous amounts of physical head movement; choosing badly (e.g., pure arrival order) can make the head thrash back and forth across the disk unnecessarily.

One-line definition: Disk scheduling decides the order in which pending I/O requests (each targeting a specific cylinder/track) are serviced, to minimize total head movement and access time.

2. Disk Geometry — the vocabulary these algorithms assume



- **Platter** — a spinning circular disk coated with magnetic material.
- **Track** — one of many concentric circles on a platter's surface.
- **Sector** — a track is divided into small fixed-size arcs; the sector is the smallest unit the disk reads/writes.
- **Cylinder** — the same track number across *all* platters/surfaces, stacked — since all heads move together, positioning at "cylinder N" positions every head at track N of its own surface simultaneously.

The three components of disk access time

Total access time = Seek time + Rotational latency + Transfer time

- **Seek time** — time to move the head to the correct **cylinder**. This is the dominant, most expensive component, and the one disk scheduling algorithms specifically target.
- **Rotational latency** — time waiting for the platter to spin so the correct **sector** rotates under the head.
- **Transfer time** — time to actually read/write the data once positioned correctly.

Why disk scheduling only targets seek time: rotational latency is largely outside useful OS control (it depends on where the platter happens to be spinning to at that instant), and transfer time is essentially fixed once positioned. Seek time, on the other hand, depends entirely on the **order** requests are serviced in — exactly the variable the OS controls.

3. Disk Scheduling Algorithms

All examples below use a common setup: **cylinders 0–199**, current head position = **50**, pending requests (in arrival order): **98, 183, 37, 122, 14, 124, 65, 67**.

3.1 FCFS (First Come First Served)

Service requests in the exact order they arrived — no reordering at all.

50 → 98 → 183 → 37 → 122 → 14 → 124 → 65 → 67

Total head movement = $|98-50| + |183-98| + |37-183| + |122-37| + |14-122| + |124-14| + |65-124| + |67-65|$

= $48 + 85 + 146 + 85 + 108 + 110 + 59 + 2 = \mathbf{643 \text{ cylinders}}$

-  Simple, fair in arrival order (same rationale as FCFS in CPU scheduling).
-  Ignores physical position entirely — can produce wildly inefficient, back-and-forth head movement, exactly as seen above.

3.2 SSTF (Shortest Seek Time First)

Always service whichever **pending** request is physically **closest** to the head's current position.

50 → 65 → 67 → 37 → 14 → 98 → 122 → 124 → 183

Total head movement = $|65-50| + |67-65| + |37-67| + |14-37| + |98-14| + |122-98| + |124-122| + |183-124|$

= $15 + 2 + 30 + 23 + 84 + 24 + 2 + 59 = \mathbf{239 \text{ cylinders}}$

-  Much better than FCFS in this example — greedily minimizes each individual seek.
-  **Starvation risk** — a request far from the current cluster of activity can be repeatedly passed over if closer requests keep arriving, the same fundamental issue as SJF in CPU scheduling (this

is essentially "SJF, but for seek distance instead of burst time," and it inherits SJF's exact weakness).

3.3 SCAN ("Elevator" algorithm)

The head moves in **one direction**, servicing every request in its path, until it reaches the **end of the disk** — then reverses direction and does the same on the way back. Behaves exactly like a building elevator: it doesn't reverse just because someone below wants to go up while it's already heading up.

Assume head is moving toward 0 first:

```
50 → 37 → 14 → 0 → 65 → 67 → 98 → 122 → 124 → 183
```

Total head movement = $(50-0) + (183-0) = 50 + 183 = \mathbf{233 \text{ cylinders}}$ (head travels all the way to 0, then all the way to 183)

- ✓ No starvation — every request is guaranteed service within, at most, one full sweep.
- ✓ Generally solid, predictable performance.
- ✗ Travels to the **very end** of the disk even if no requests are pending out there, wasting some movement (visible above — the head goes all the way to cylinder 0, even though the closest request in that direction was only at 14).

3.4 C-SCAN (Circular SCAN)

Like SCAN, but instead of reversing direction at the end, the head jumps back to the **beginning** of the disk and starts another sweep in the **same** direction — treating the disk as **circular**, so wait times are more uniform.

Assume the head is moving toward 199 first (switching direction from the SCAN/LOOK examples above, just to show the pattern the other way):

```
50 → 65 → 67 → 98 → 122 → 124 → 183 → 199 → (jump) → 0 → 14 → 37
```

- **Why go all the way to 199 and jump to 0, rather than just reversing?** SCAN's reversal means a request that *just missed* the head near the end of one pass gets serviced almost immediately (head reverses right back toward it), while a request that just missed it near the *start* of a pass waits almost a full sweep — **uneven wait times**. C-SCAN makes every request wait a comparable amount by *always* sweeping in the same direction, giving genuinely uniform wait-time distribution across the disk.
- ✗ The jump back to the beginning is "wasted" movement (no requests serviced during the jump itself), but this cost is deliberately accepted in exchange for fairness/uniformity.

3.5 LOOK

Identical to SCAN, **except** the head only travels as far as the **last request** in its current direction, then immediately reverses — it doesn't needlessly continue to the physical end of the disk if nothing is waiting out there.

50 → 37 → 14 → 65 → 67 → 98 → 122 → 124 → 183

Total head movement = $(50-14) + (183-14) = 36 + 169 = \mathbf{205 \text{ cylinders}}$

- ☒ Strictly better than SCAN in practice — same fairness/no-starvation benefit, without wasting movement on empty regions of the disk.

3.6 C-LOOK

The circular equivalent of LOOK — like C-SCAN, but only travels as far as the last request before jumping back, rather than all the way to the physical end.

50 → 65 → 67 → 98 → 122 → 124 → 183 → (jump) → 14 → 37

- Combines C-SCAN's uniform-wait-time fairness with LOOK's "don't travel further than necessary" efficiency — generally considered one of the best practical choices among these.

Summary Comparison

Algorithm	Reorders by	Starvation risk	Wastes movement on empty disk regions?	Wait-time uniformity
FCFS	Arrival order (no reordering)	No	N/A	Poor, unpredictable
SSTF	Closest request	Yes	No	Poor (favors the current cluster)
SCAN	Direction sweep, full disk	No	Yes (goes to physical end)	Uneven (edge effect)
C-SCAN	Direction sweep, full disk, circular	No	Yes	Uniform
LOOK	Direction sweep, last request only	No	No	Uneven (edge effect)
C-LOOK	Direction sweep, last request only, circular	No	No	Uniform

Interview soundbite: "SSTF is the disk-scheduling analogue of SJF — greedy and efficient on average, but starvation-prone. SCAN/LOOK behave like an elevator, sweeping in one direction to guarantee every request eventually gets serviced. The 'C' (circular) variants trade a bit of extra movement (jumping back to the start) for genuinely uniform wait times across the whole disk, rather than penalizing requests near the edges of a sweep. LOOK/C-LOOK refine SCAN/C-SCAN further by not traveling past the last actual pending request."

4. A Note on SSDs

Everything above assumes a **mechanical** disk, where physical head movement is the dominant cost. **Solid-state drives (SSDs)** have no moving parts and no meaningful "seek time" — any location can be accessed in roughly constant time (though there are still SSD-specific realities like wear-leveling and write amplification, which are a different topic entirely). This means classic seek-minimizing algorithms like SCAN/LOOK provide little to no benefit on an SSD — modern OSes typically use a much simpler scheduler (or none at all — e.g., Linux's "none"/"noop" I/O scheduler) for SSD-backed storage, since there's no physical seek cost left to optimize away.

Interview soundbite: "SCAN and friends are solving a specifically mechanical problem — minimizing physical head travel. On an SSD, there's no head and no seek time, so that entire class of optimization becomes largely irrelevant; the OS can use a much simpler I/O scheduler instead."

Interview Questions With Answers

Q1. What are the three components of disk access time, and which one does disk scheduling actually target?

Answer: Seek time (moving the head to the correct cylinder), rotational latency (waiting for the right sector to rotate under the head), and transfer time (actually reading/writing once positioned). Disk scheduling specifically targets **seek time**, because it's the dominant cost and the only one that depends on the *order* requests are serviced in — rotational latency depends on the platter's current spin position (largely outside useful control), and transfer time is essentially fixed once positioned.

Q2. Why is SSTF described as the disk-scheduling analogue of SJF, and what weakness does it inherit?

Answer: Just as SJF always picks the ready process with the shortest burst time, SSTF always picks the pending request closest to the head's current position — both are greedy, locally-optimal choices. SSTF inherits SJF's exact weakness: starvation. A request far from wherever the head currently is can be repeatedly skipped over if a cluster of closer requests keeps arriving, just as a long CPU burst can be repeatedly skipped in favor of a stream of shorter ones under SJF.

Q3. How does SCAN prevent the starvation that SSTF suffers from?

Answer: SCAN sweeps the head in one direction, servicing every request in its path, all the way to the end of the disk, before reversing. Because it commits to servicing everything in its current direction rather than always jumping to whatever's closest, no request can be indefinitely skipped — the absolute worst case for any request is waiting for the current sweep to reach the far end and come back, which is a bounded amount of time, not an unbounded one.

Q4. Why does C-SCAN jump back to the beginning instead of just reversing direction like SCAN?

Answer: SCAN's direction-reversal creates uneven wait times — a request that just missed the head near the *end* of a sweep gets serviced almost immediately once the head reverses, while a request near the *start* of a sweep has to wait almost a full sweep. By always moving in the same direction and jumping back to the start once it reaches the end, C-SCAN makes the wait time comparable for requests anywhere on the disk, rather than favoring positions near where the head happens to reverse.

Q5. What's the difference between SCAN and LOOK?

Answer: SCAN always travels all the way to the physical end of the disk before reversing, even if there are no pending requests out there. LOOK "looks ahead" and only travels as far as the last actual pending request in the current direction before reversing — avoiding wasted movement through empty regions of the disk that SCAN would otherwise traverse unnecessarily.

Q6. Why is C-LOOK generally considered one of the best practical choices among these algorithms?

Answer: It combines both good properties simultaneously: like C-SCAN, it always sweeps in the same direction (jumping back to the start rather than reversing), giving uniform wait times across the whole disk; and like LOOK, it only travels as far as the last actual pending request rather than to the disk's physical end, avoiding wasted movement. It gets fairness without the extra wasted-movement cost that plain SCAN/C-SCAN pay near the disk's edges.

Q7. Why don't SCAN/LOOK-style algorithms provide meaningful benefit on SSDs?

Answer: These algorithms are specifically designed to minimize *physical head seek time* on a mechanical disk. SSDs have no moving read/write head and no meaningful seek time at all — any location can be accessed in roughly constant time — so there's no physical positioning cost left for a scheduling algorithm to optimize away. Using a complex seek-minimizing scheduler on an SSD adds scheduling overhead for essentially zero benefit, which is why simpler I/O schedulers are typically used for SSD-backed storage instead.

Q8. Scenario: A disk has heavy, continuous request traffic clustered around cylinders 40-60, with the head starting at cylinder 50. A single request also arrives for cylinder 195. Compare how SSTF and LOOK would treat this lone distant request.

Answer: Under SSTF, the request at cylinder 195 could be starved indefinitely — as long as new requests keep arriving in the 40-60 cluster (all much closer to the head than 195), SSTF will always greedily pick one of those instead, and the distant request may never get serviced while that traffic continues. Under LOOK, once the head's current sweep in that direction reaches the end of the pending requests in the 40-60 cluster, it would still need to continue toward 195 (since that request is

now the furthest pending one in that direction) before reversing — guaranteeing the distant request gets serviced within one sweep, no matter how much clustered traffic exists elsewhere, because LOOK commits to fully covering one direction's requests rather than always chasing the closest one.

Q9. Why is FCFS considered a poor disk scheduling choice despite being simple and "fair"?

Answer: FCFS services requests purely in arrival order with no regard to their physical position on the disk, which can force the head to move back and forth erratically across large distances if requests happen to arrive in a physically scattered order — exactly as shown in the worked example, where FCFS produced dramatically more total head movement (643 cylinders) than SSTF (239) or LOOK (205) for the identical request set. Its "fairness" (arrival order) doesn't translate into efficient physical movement, which is the actual metric disk scheduling cares about.

Q10. What general design principle do SCAN, C-SCAN, LOOK, and C-LOOK all share, and how does it differ from SSTF's principle?

Answer: All four "elevator-style" algorithms commit to a *systematic sweep direction*, servicing requests as the head passes them, rather than re-evaluating "what's closest" after every single request like SSTF does. This directional commitment is exactly what guarantees no request can be starved — every request is guaranteed to eventually be in the head's path during some sweep, which is not true of SSTF's purely greedy, closest-first approach.